三种地址映射方式下，何时触发替换算法

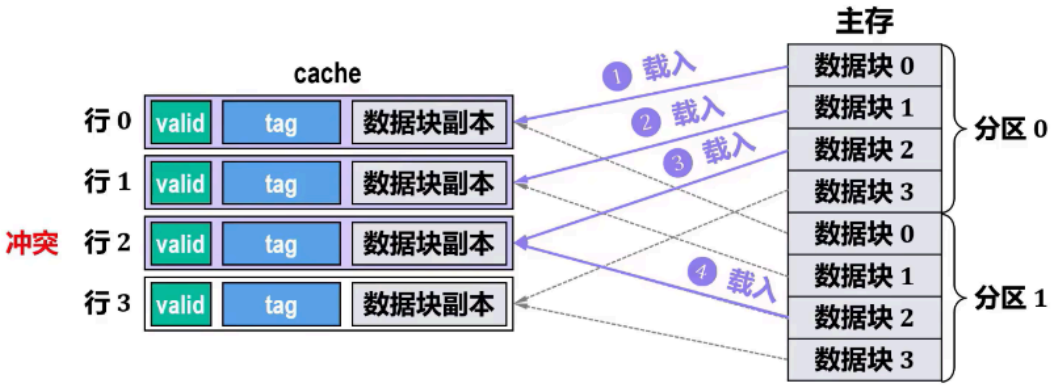
1.1 直接映射

cache中主存数据块的替换（淘汰）算法

何时使用替换算法　cache未满时就有可能出现冲突，需要进行cache中主存数据块的替换

常用的替换算法　由于每个主存数据块只对应一个固定的cache行，因此不需要使用任何替换算法

直接映射方式举例



1.2 全相联映射

cache中主存数据块的替换（淘汰）算法

何时使用替换算法　cache已满后才有可能出现冲突，需要进行cache中主存数据块的替换

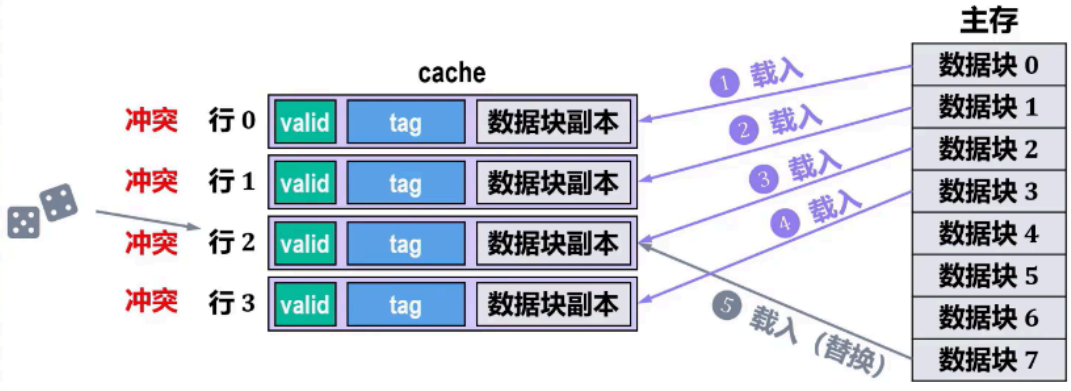
常用的替换算法　随机替换

优点：硬件实现最容易，速度也最快。

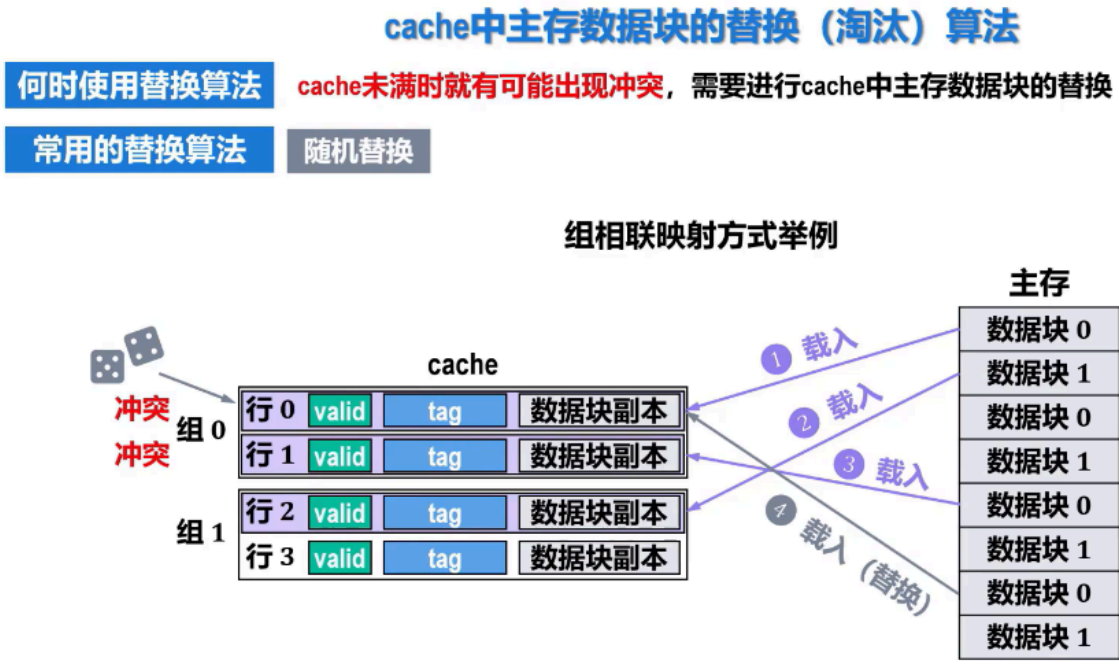
缺点：随机替换掉的数据可能马上又要用，从而降低命中率和cache的工作效率。

- 随着cache容量的增大，随机替换算法的缺点所带来的负面影响会减小。
- 通过仿真试验可发现，随机替换算法的性能只稍逊于LFU算法和LRU算法。

全相联映射方式举例

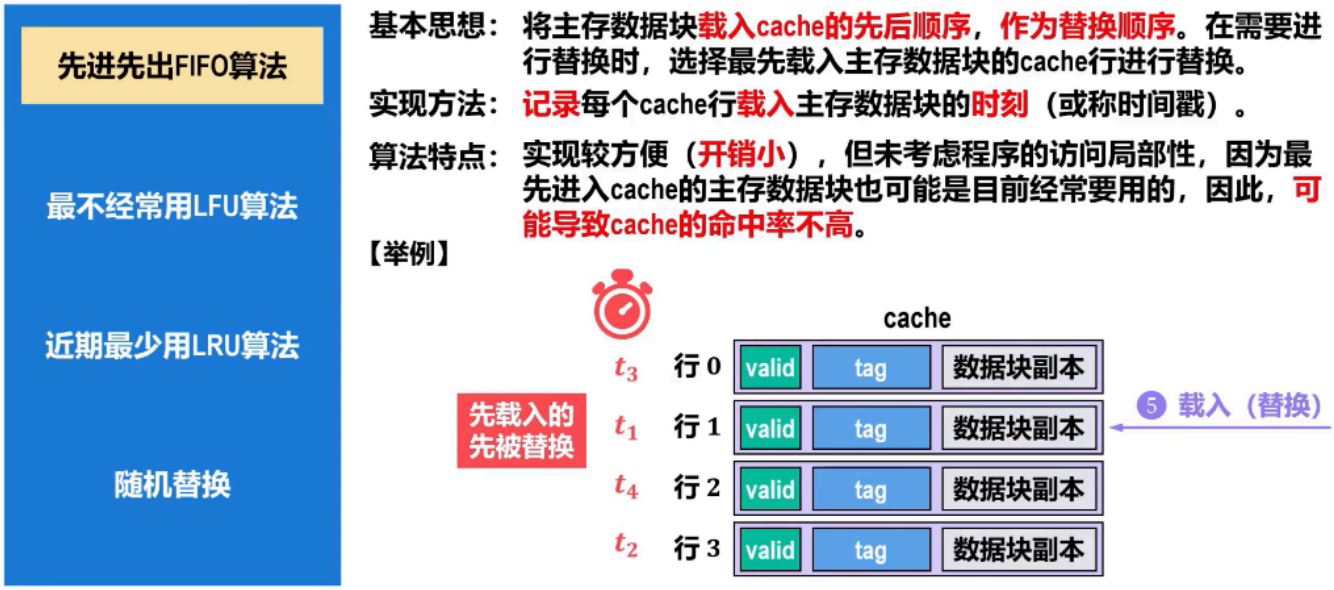


1.3 组相联映射



二、替换算法

2.1 先进先出FIFO



2.2 最不经常使用LFU



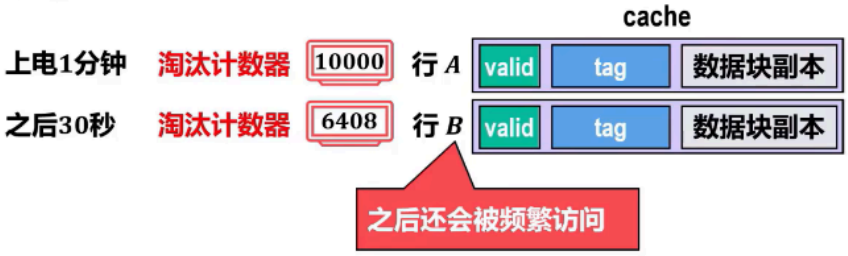
基本思想：将被命中访问次数最少的cache行淘汰。

实现方法：需要为每个cache行设置一个淘汰计数器，新载入的cache行从0开始计数，每命中访问1次，被访问cache行的淘汰计数器加1。需要进行替换时，计数值最小的cache行被替换。

算法特点：硬件成本较高（每行一个计数器）

淘汰计数器从cache上电后就开始记录，但不能严格反映出近期的访问情况。

【举例】



2.3 最近最少使用LRU



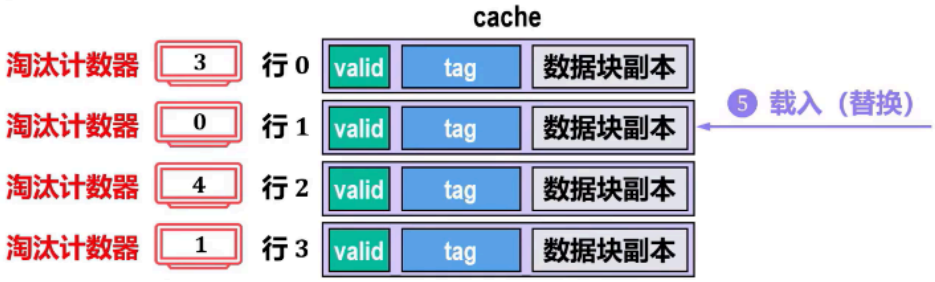
基本思想：将近期内最少被命中访问过的cache行淘汰。

实现方法：与LFU算法类似，也需要为每个cache行设置一个淘汰计数器。不同点是：cache行每被访问（载入/替换、命中）1次，该cache行的计数器被清零，其他计数值不为空的cache行各自的计数器被加1。需要进行替换时，计数值最大的cache行被替换。

算法特点：硬件成本较高（每行一个计数器）

比较正确地反映了程序的访问局部性，因为近期最少使用的主存数据块，很可能是将来最少被访问的。

【举例】



LRU位

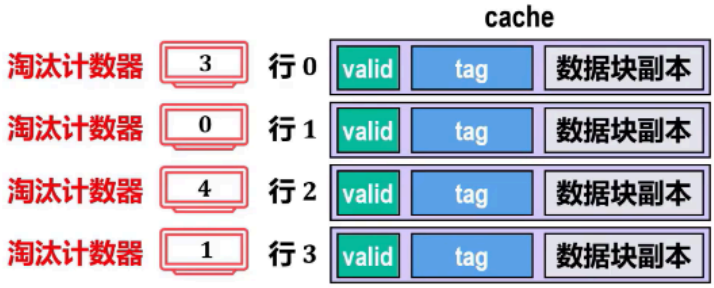
先进先出FIFO算法

最不经常用LFU算法

近期最少用LRU算法

随机替换

本例中，LRU位的位数为$\log_2 4 = 2$，取值范围0~3。



- 淘汰计数器的数值，称为LRU位，其位数与cache组（当只有1个组时，则为全相联映射）的大小有关。
  - 理论上，n路组相联（即每个cache组包含cache行的数量为n）时LRU位的位数为$\log_2 n$，例如，2路组相联时LRU位的位数为1，4路组相联时LRU位的位数为2。

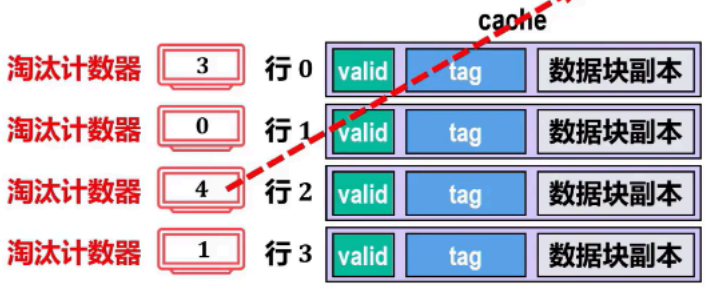
先进先出FIFO算法

最不经常用LFU算法

近期最少用LRU算法

随机替换

本例中，LRU位的位数为$\log_2 4 = 2$，取值范围0~3。矛盾



LRU算法规则：cache行每被访问（载入/替换、命中）1次，该行的计数器被清零，其他计数值不为空的行、各自的计数器被加1。需要进行替换时，计数值最大的行被替换。

改为：命中时，该行的计数器被清零，比其计数值小的其他行的计数器加1。这样可满足LRU位的位宽限制。

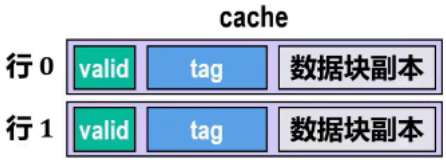
减少同学们需要记忆的规则数量

先进先出FIFO算法

最不经常用LFU算法

近期最少用LRU算法

随机替换



- 如果是仅包含2个cache行的全相联映射（或2路组相联映射），由于1个主存数据块只能在这2个cache行存放，2选1完全不需要淘汰计数器，仅需要1个二进制标志位即可。例如：当行0载入新的主存数据块时，将标志位置1，而行1载入新的主存数据块时将标志位清零。这样，当需要进行替换时，只需要检查该标志位即可：
  - 若标志位为0，则说明行1的主存数据块比行0的后载入，因此应替换行0的主存数据块，否则对行1进行替换。